

Algoritmos y Estructuras de Datos III

Trabajo Práctico N° 5	Unidad 4
Modalidad: Semi -Presencial	Estratégica Didáctica: Trabajo individual
Metodología de Desarrollo: Det. docente	Metodología de Corrección: Via Classroom.
Carácter de Trabajo: Obligatorio – Con Nota	Fecha Entrega: A confirmar por el Docente.

ÁRBOLES

MARCO TEÓRICO:

Responder el siguiente cuestionario en función de la bibliografía Obligatoria.

1. ¿Qué es un Grafo?

Un grafo es como una estructura que tiene muchos puntos o nodos conectados. Estos puntos se llaman vértices y las conexiones entre ellos se llaman aristas. Un grafo se puede representar como una combinación de estos vértices y aristas.

Los grafos pueden ser de diferentes tipos. Por ejemplo, un grafo dirigido tiene aristas que van en una dirección específica, como una calle de un solo sentido. Un grafo no dirigido tiene aristas que no tienen dirección, así que puedes ir en ambos sentidos. También hay grafos mixtos que combinan estos dos tipos.

Hay grafos que tienen pesos o costos en las aristas, como la distancia entre ciudades conectadas por carreteras. Cuando dos nodos están conectados por una arista, se llaman nodos adyacentes.

Si hay más de una arista entre dos nodos, se llaman aristas paralelas y el grafo se llama multigrafo. Un nodo que no está conectado a ningún otro se llama nodo aislado. Un grafo que solo tiene nodos aislados se llama grafo nulo.

Los árboles son como grafos especiales. Son grafos conectados que no tienen dirección y no tienen ciclos. Esto significa que no puedes empezar en un punto y dar vueltas para regresar al mismo punto.

2. ¿Qué Árbol y qué tipos de Problemas resuelve?

Un árbol es una forma de organizar datos de manera jerárquica, es decir, no lineal. A diferencia de las estructuras lineales como las listas y los arrays, donde cada elemento solo tiene un elemento antes y uno después, en un árbol cada elemento puede tener varios elementos después de él, llamados hijos, pero solo uno antes, llamado padre. La excepción es la raíz, que es el elemento principal y no tiene padre.

Los elementos importantes en un árbol son la raíz, que es el punto de entrada, los nodos internos, que tienen un padre y hijos, las hojas, que son los nodos que no tienen hijos, y las aristas, que son las conexiones entre los padres y los hijos.

Los árboles son útiles para organizar información de manera jerárquica, como en los sistemas de archivos y las estructuras organizacionales. También son útiles para buscar y ordenar datos de manera eficiente, representar expresiones matemáticas y lógicas, indexar bases de datos y comprimir datos. La principal ventaja de los árboles es que permiten acceso, inserción y eliminación de datos en un tiempo mucho más rápido que las estructuras lineales, es decir, en tiempo logarítmico en lugar de lineal. Esto significa que los árboles pueden manejar grandes cantidades de datos de manera más eficiente que las estructuras lineales.

3. ¿Qué función tiene un Árbol Binario?. ¿Qué operaciones posee?

Un árbol binario es un tipo de árbol en el que cada parte tiene como máximo dos hijos: uno a la izquierda y otro a la derecha. La forma más común de árbol binario es el árbol binario de búsqueda. En este tipo de árbol, cada parte tiene una característica especial: todos los valores que están a la izquierda de la parte son menores que la parte misma, y todos los valores que están a la derecha son mayores. Esto hace que encontrar cosas en el árbol sea muy eficiente.

En el código que se usa en el laboratorio, el árbol binario se hace con dos clases: `NodoArbol` y `ArbolBinario`. La clase `NodoArbol` tiene el dato que se va a guardar, y también tiene punteros para el hijo izquierdo y el hijo derecho, que al principio están vacíos. La clase `ArbolBinario` tiene un puntero que apunta al principio del árbol, que se llama raíz.

El árbol binario puede hacer varias cosas, como:

- Crear un árbol: se hace de manera recursiva, creando cada parte del árbol una por una, y preguntando si cada parte tiene hijos a la izquierda o a la derecha.
- Buscar algo en el árbol: se compara lo que se busca con cada parte del árbol, yendo a la izquierda si es menor o a la derecha si es mayor, hasta que se encuentra o se llega a un punto final.
- Agregar algo nuevo al árbol: es similar a buscar, pero cuando se encuentra el lugar correcto, se agrega la nueva parte como una hoja.
- Eliminar algo del árbol: hay tres casos diferentes - si la parte que se va a eliminar no tiene hijos, se puede eliminar directamente; si tiene un hijo, se reemplaza con ese hijo; y si tiene dos hijos, se reemplaza con el sucesor, que es el menor de los que están a la derecha.

Recorrer el árbol: hay tres formas de hacer esto, que se llaman preorden, inorden y postorden.

- El recorrido en preorden sigue este orden: primero la raíz, luego la parte izquierda y finalmente la parte derecha. Se usa para copiar el árbol o convertirlo en una forma que se pueda guardar.
- El recorrido en inorden sigue este orden: primero la parte izquierda, luego la raíz y finalmente la parte derecha. En un árbol binario de búsqueda, esto hace que los datos salgan en orden ascendente.
- El recorrido en postorden sigue este orden: primero la parte izquierda, luego la parte derecha y finalmente la raíz. Se usa para liberar la memoria que usa el árbol o para evaluar expresiones.

4. ¿Qué es un Árbol Balanceado, dar ejemplos de su uso?

Un árbol balanceado es un árbol binario de búsqueda donde la diferencia de altura entre el subárbol izquierdo y el derecho de cualquier nodo no es mayor que un valor específico, generalmente 1.

Esto asegura que el árbol no se convierta en una lista lineal. Esto es algo que podría pasar si se insertan datos ordenados en un árbol binario de búsqueda normal. En ese caso, todas las búsquedas tardarían mucho, $O(n)$.

El árbol AVL es un tipo común de árbol balanceado. En él, la diferencia de altura entre el subárbol izquierdo y el derecho de cada nodo debe estar entre -1 y 1.

Si se inserta o elimina un nodo y se rompe esta regla, el árbol se ajusta mediante rotaciones. Estas rotaciones pueden ser simples o dobles, hacia la izquierda o hacia la derecha.

Los árboles balanceados se usan en muchas aplicaciones. Algunos ejemplos son:

- Motores de bases de datos para índices que necesitan mantenerse balanceados.
- Compiladores para tablas de símbolos.
- Sistemas de archivos que requieren acceso rápido.
- Cualquier aplicación donde las búsquedas deban ser rápidas, sin importar el orden en que se insertan los datos.

5. ¿Qué es un Árbol B y B+, cual es la diferencia?

ARBOL B

El árbol B es una forma de árbol que se utiliza para organizar datos de manera eficiente. Este árbol es una versión mejorada del árbol binario balanceado, pero en lugar de tener solo dos hijos por nodo, el árbol B puede tener múltiples hijos.

Un árbol B de grado n tiene algunas propiedades importantes. La raíz del árbol puede almacenar entre 1 y $2n$ datos. Los nodos internos y las hojas del árbol pueden almacenar entre n y $2n$ datos. Los nodos internos tienen entre $n+1$ y $2n+1$ hijos. Todas las hojas del árbol tienen la misma altura, lo que significa que la distancia desde la raíz hasta cualquier hoja es la misma. La información en el árbol está ordenada, lo que facilita la búsqueda de datos. Tanto los nodos internos como las hojas almacenan datos completos.

El árbol B admite varias operaciones. La búsqueda es una de ellas. Cuando se busca un dato, el árbol compara el dato con los valores del nodo y baja por el hijo correspondiente hasta encontrar el dato.

La inserción es otra operación. Cuando se inserta un nuevo dato, el árbol localiza la hoja donde insertar el dato. Si la hoja está llena, el árbol la divide y sube el elemento del medio al padre. Esto puede propagarse hasta la raíz del árbol, lo que aumenta la altura del árbol.

La eliminación es la tercera operación. Cuando se elimina un dato, el árbol quita el dato. Si el nodo queda con menos de n elementos, el árbol puede robar elementos del hermano o fusionar páginas, lo que puede reducir la altura del árbol.

ARBOL B+

Es una especie de árbol B, pero con una gran diferencia. En el árbol B+, la información completa solo se guarda en las hojas. Los nodos internos y la raíz solo guardan claves o índices que nos ayudan a encontrar los datos.

Cuando encontramos una clave en un nodo interno, no podemos detenernos ahí. Tenemos que seguir buscando hacia abajo hasta encontrar el dato en la hoja correcta. Otra cosa que distingue al árbol B+ es que sus hojas están conectadas entre sí. Esto permite hacer búsquedas secuenciales de manera eficiente.

La principal diferencia entre el árbol B y el B+ es dónde se encuentran los datos. En el árbol B, los datos pueden estar en cualquier nivel y la búsqueda termina cuando los encontramos. En el árbol B+, la búsqueda siempre termina en una hoja. Las claves en los nodos internos son solo copias que nos sirven de guía.

El árbol B+ es muy utilizado en bases de datos porque sus hojas enlazadas permiten hacer búsquedas por rangos de manera muy eficiente.

MARCO PRÁCTICO

1. Ejercicio 1. Construye un árbol binario de búsqueda a partir de la siguiente lista: G B Q A C K F P D E R H
2. Ejercicio 2. Para el árbol obtenido en el ejercicio 1 muestra el estado del mismo tras extraer de forma iterativa los siguientes elementos: a) E b) C c) G
3. Ejercicio 3. Para el árbol obtenido en el ejercicio 1 muestra la evolución de los siguientes algoritmos de recorrido de árboles binarios:
 - A. Recorrido en preorden
 - B. Recorrido en inorden
 - C. Recorrido en postorden

https://github.com/FernandezPegoRocio/-Algoritmos_y_Estructuras-de-Datos_III